

CO5-AT2

DSA0103-Object Oriented Programming using C++

Name: V. Reddy Sreya

Regno: 192525124

1. Develop a Banking Management System that supports Savings, Current, and Loan Accounts. The system must allow users to perform operations such as account creation, deposit, withdrawal, balance enquiry, and loan processing.(CO5)

Requirements:

Define an abstract base class Account with virtual and pure virtual functions for common operations (deposit, withdraw, displayBalance, getAccountType).

2. Use runtime polymorphism (base class pointers/references) so the correct accountspecific behavior executes based on the actual object type.
3. Handle errors using exception handling insufficient balance, invalid account number, negative deposit, and invalid withdrawal.
4. Demonstrate throwing, catching, and rethrowing exceptions during a transaction (e.g., a failed fund transfer).
5. Use exception specifications (throw(...)) on functions to declare the exceptions they may raise.

Requirements:

Define an abstract base class Account with virtual and pure virtual functions for common operations:

deposit

withdraw

displayBalance

getAccountType

Use runtime polymorphism (base class pointers/references) so the correct account-specific behavior executes based on the actual object type.

Handle errors using exception handling:

Insufficient balance

Invalid account number

Negative deposit

Invalid withdrawal

Demonstrate throwing, catching, and rethrowing exceptions during a transaction, for example, a failed fund transfer.

Use exception specifications throw(...) on functions to declare the exceptions they may raise.

Code:

SIMATS Online Programming Lab

C++ PROGRAMMING

Vadlathala Reddy Sreya
192525124RA

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 class Account {
6 protected:
7     int no;
8     double balance;
9 public:
10     Account(int n, double b) : no(n), balance(b) {}
11     virtual void deposit(double a) = 0;
12     virtual void withdraw(double a) = 0;
13     virtual void displayBalance() = 0;
14     virtual string getAccountType() = 0;
15     virtual ~Account() {}
16 };
17
18 class Savings : public Account {
19 public:
20     Savings(int n, double b) : Account(n,b) {}
21
```

Your INPUT goes here!

SIMATS Online Programming Lab

C++ PROGRAMMING

Vadlathala Reddy Sreya
192525124RA

Your INPUT goes here!

SIMATS Online Programming Lab

C++ PROGRAMMING

Vadlathala Reddy Sreya
192525124RA

```
43 public:
44     Current(int n, double b) : Savings(n,b) {}
45
46     string getAccountType() {
47         return "Current";
48     }
49 };
50
51 class Loan : public Savings {
52 public:
53     Loan(int n, double b) : Savings(n,b) {}
54
55     string getAccountType() {
56         return "Loan";
57     }
58 };
59
60 void transfer(Account *a, Account *b, double amount) {
61     try {
62         a->withdraw(amount);
63         b->deposit(amount);
```

Your INPUT goes here!

SIMATS Online Programming Lab

C++ PROGRAMMING

Vadlathala Reddy Sreya
192525124RA

```
64         cout << "Transfer Successful\n";
65     }
66     catch(...) {
67         cout << "Transfer Failed\n";
68         throw;
69     }
70 }
71
72 int main() {
73     Account *a = new Savings(101, 5000);
74     Account *b = new Current(102, 3000);
75
76     try {
77         cout << a->getAccountType() << endl;
78
79         a->deposit(1000);
80         a->withdraw(500);
81         a->displayBalance();
82
83         transfer(a, b, 1000);
84     }
```

Your INPUT goes here!

SIMATS Online Programming Lab

C++ PROGRAMMING

Vadlathala Reddy Sreya
192525124RA

```
85     catch(string e) {
86         cout << "Error: " << e << endl;
87     }
88
89     delete a;
90     delete b;
91     return 0;
92 }
```

Your INPUT goes here!

Output:

SIMATS Online Programming Lab

C++ PROGRAMMING

 Vadlathala Reddy Sreya
192525124RA

93

94

OUTPUT

```
Savings
Balance: 5500
Transfer Successful
```

2. Develop an Online Payment System for an e-commerce application that supports Credit Card, Debit Card, UPI, and Net Banking payments. The system should process different payment methods and handle transaction failures safely. (COS)

Requirements:

1. Define an abstract base class `PaymentMethod` with virtual and pure virtual functions forming a common payment interface (`validateDetails`, `pay`, `getMethodName`).
2. Use runtime polymorphism to dynamically select and execute the correct payment method's logic at runtime.
3. Handle errors using exception handling - invalid card details, insufficient funds, incorrect UPI PIN, and failed transactions.
4. Demonstrate throwing, catching, and rethrowing exceptions during payment processing (e.g., in a payment gateway's checkout flow).
5. Use exception specifications (`throw(..)`) to declare possible payment-related exceptions on each function.

Define an abstract base class `PaymentMethod` with virtual and pure virtual functions forming a common payment interface:

`validateDetails`

`pay`

`getMethodName`

Use runtime polymorphism to dynamically select and execute the correct payment method's logic at runtime.

Handle errors using exception handling:

Invalid card details

Insufficient funds

Incorrect UPI PIN

Failed transactions

Demonstrate throwing, catching, and rethrowing exceptions during payment processing, for example, in a payment gateway's checkout flow.

Use exception specifications `throw(...)` to declare possible payment-related exceptions on each function.

Display appropriate success or error messages based on the transaction outcome.

6. Display appropriate success or error messages based on the transaction outcome.

Code:



```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 class PaymentMethod {
6 public:
7     virtual void validateDetails() = 0;
8     virtual void pay(double) = 0;
9     virtual string getMethodName() = 0;
10    virtual ~PaymentMethod() {}
11 };
12
13 class Card : public PaymentMethod {
14 public:
15     void validateDetails() {
16         cout << "Card Validated\n";
17     }
18
19     void pay(double amount) {
20         if(amount > 50000)
```

Your INPUT goes
here!



```
21         throw string("Insufficient Funds");
22         cout << "Card Payment: Rs." << amount << endl;
23     }
24
25     string getMethodName() {
26         return "Credit/Debit Card";
27     }
28 };
29
30 class UPI : public PaymentMethod {
31 public:
32     void validateDetails() {
33         cout << "UPI Validated\n";
34     }
35
36     void pay(double amount) {
37         if(amount <= 0)
38             throw string("Transaction Failed");
39         cout << "UPI Payment: Rs." << amount << endl;
40     }
```

Your INPUT goes
here!

SIMATS Online Programming Lab

C++ PROGRAMMING



Vadlathala Reddy Sreya
192525124RA



```
41
42     string getMethodName() {
43         return "UPI";
44     }
45 };
46
47 class NetBanking : public PaymentMethod {
48 public:
49     void validateDetails() {
50         cout << "Net Banking Validated\n";
51     }
52
53     void pay(double amount) {
54         if(amount > 100000)
55             throw string("Insufficient Funds");
56         cout << "Net Banking Payment: Rs." << amount << endl;
57     }
58
59     string getMethodName() {
60         return "Net Banking";
```

Your INPUT goes
here!

SIMATS Online Programming Lab

C++ PROGRAMMING



Vadlathala Reddy Sreya
192525124RA



```
61     }
62 };
63
64 void process(PaymentMethod *p, double amount) {
65     try {
66         p->validateDetails();
67         p->pay(amount);
68         cout << "Transaction Successful\n";
69     }
70     catch(...) {
71         cout << "Payment Failed\n";
72         throw;
73     }
74 }
75
76 int main() {
77     PaymentMethod *p;
78
79     int choice;
80     double amount;
```

Your INPUT goes
here!

SIMATS Online Programming Lab

C++ PROGRAMMING

Vadlathala Reddy Sreya
192525124RA

```
81
82  cout << "1. Card\n2. UPI\n3. Net Banking\n";
83  cin >> choice;
84
85  if(choice == 1)
86      p = new Card();
87  else if(choice == 2)
88      p = new UPI();
89  else
90      p = new NetBanking();
91
92  cout << "Enter amount: ";
93  cin >> amount;
94
95  try {
96      cout << p->getMethodName() << endl;
97      process(p, amount);
98  }
99  catch(string e) {
100      cout << "Error: " << e << endl;
```

Your INPUT goes here!

SIMATS Online Programming Lab

C++ PROGRAMMING

Vadlathala Reddy Sreya
192525124RA

```
101  }
102
103  delete p;
104  return 0;
105 }
```

Output:

SIMATS Online Programming Lab

C++ PROGRAMMING

Vadlathala Reddy Sreya
192525124RA

```
106
107 |
```

OUTPUT

```
1. Card
2. UPI
3. Net Banking
Enter amount: Net Banking
Net Banking Validated
Net Banking Payment: Rs.6.95156e-310
Transaction Successful
```